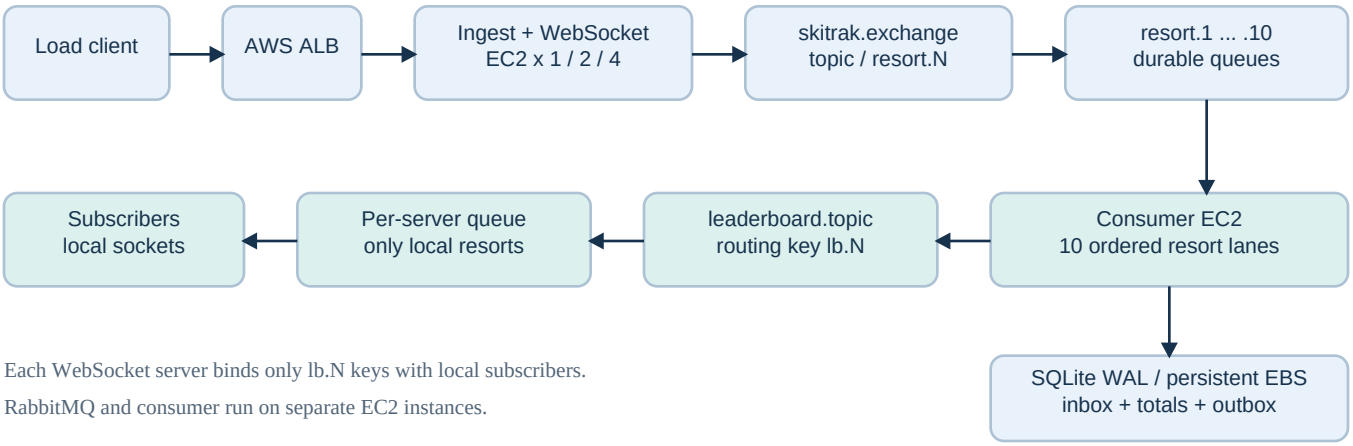
SkiTrak | Assignment 2

Architecture 1/2 - deployment, topology and connection locality



Each WebSocket server binds only lb.N keys with local subscribers.

RabbitMQ and consumer run on separate EC2 instances.

Routing and state ownership

Java 11/Tomcat 9 ingest servers publish persistent swipe events through one reused producer connection and a bounded, exclusive channel pool. The dedicated RabbitMQ EC2 routes skitrak.exchange topic keys resort.1 through resort.10 to ten durable queues. A separate Consumer EC2 owns one FIFO aggregation lane and one SQLite database per resort.

Subscriber-aware topic backplane

The Consumer publishes complete versioned Top-20 snapshots to leaderboard.topic using lb.{resortID}. Each WebSocket node owns an exclusive generated queue and binds only the resorts with local subscribers. This bridges connection locality without requiring resort co-location. It reduces uninterested traffic but adds binding, bootstrap and reconnect coordination.

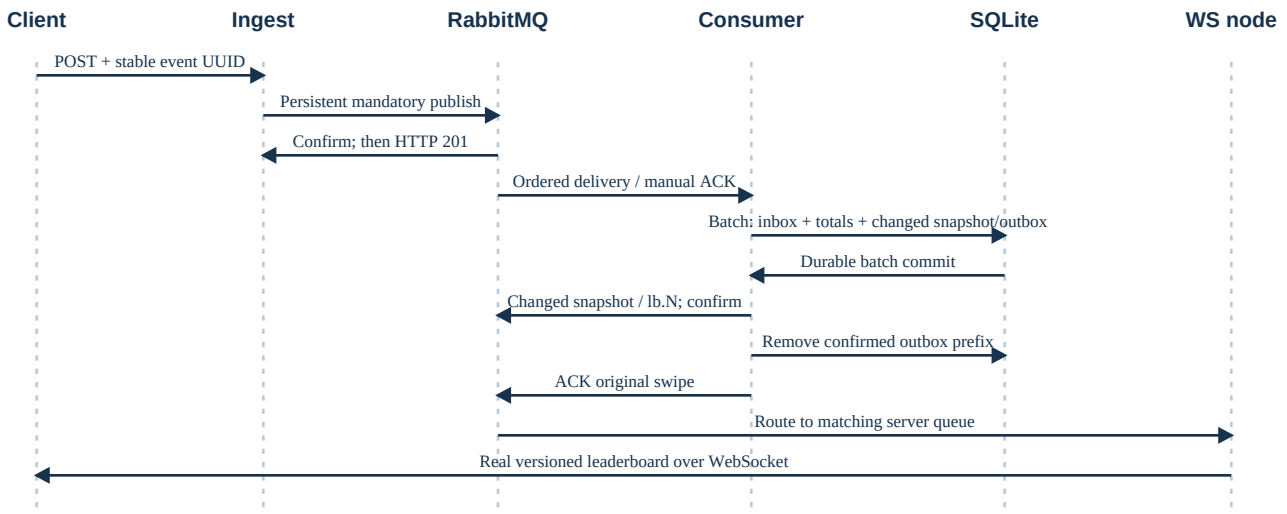
First subscriber binds before bootstrap from the authoritative Consumer snapshot. Live updates and bootstrap responses reconcile by resort/day/version. Last unsubscribe removes the binding. Broker reconnect recreates the private queue and restores bindings; periodic snapshot refresh repairs transient notification loss.

ALB and threading

ALB forwards HTTP and WebSocket upgrades to ROOT-deployed ingest apps. Health path /health: 30 s interval, 5 s timeout,healthy 2 / unhealthy 3. Idle timeout 120 s; generated-cookie stickiness 300 s. Each WebSocket stays on the node that accepted it; after failure a new connection bootstraps on another healthy node. Configurable worker pools process fair serial resort lanes; ten resorts cap useful aggregation parallelism at ten. Dynamic scaling is supported but disabled during fixed-worker comparisons.

Delivery and failure handling

Architecture 2/2 - swipe-to-leaderboard sequence



Durable aggregation and acknowledgement

A stable event UUID is reused through Idempotency-Key on retries. Ingest validates the official lift mapping and returns 201 only after mandatory routing and publisher confirmation. Uncertain outcomes return 503. Per-resort SQLite WAL/FULL transactions atomically persist dedupe inbox, skier totals and any changed snapshot/version/outbox. Batches hold at most 64 events with a 10 ms partial-batch flush trigger. Top-N changes are retained individually and published in order. Grouped broadcast confirms precede source ACKs; the confirmed outbox prefix is deleted. Prefetch 128 and ACK batch 16 are fixed for the fresh comparisons.

Recovery and boundaries

One recovery owner rebuilds a failed producer connection/channel pool; stale leases cannot re-enter its replacement. Consumer failures retry in the same lane before reconnect/redelivery. Inbox state prevents double-counting and persistent outbox state permits notification replay. Conflicting/poison events pause a resort instead of silently reordering it. systemd/Docker restart processes; persistent directories survive process restart. Source queues default to 24 h TTL, capacity 100000 and reject-publish overflow; expired events reach dead-letter queues and are not counted as successful aggregation.

The backplane is transient notification transport; the Consumer is the authoritative bootstrap source. Publisher confirm is broker acceptance, not proof that every WebSocket client received an update. Host/disk loss and multi-host aggregation failover require additional replication and ownership design. Fresh report measurements, separate from this architecture handout, disclose sampled queue peaks, clock-dependent lag means and missed performance targets.